

Front-End Technical Interview Prep

React · TypeScript · JavaScript · RxJS · Testing · Architecture

Study guide organized by job requirements · condensed answers with examples

How to use this: Each topic has a short, interview-ready answer and a code example where it helps. Bordered blue boxes are the *one-liner* you can say out loud. Yellow boxes are *gotchas* / *senior signals*. The recurring theme from feedback: **be precise and name things** — pair every concept with a concrete use case.

Contents

1. JavaScript Fundamentals JD: Strong JS
2. TypeScript JD: Strong TS
3. React JD: Strong React
4. RxJS JD: Strong RxJS
5. State Management & Data Fetching JD: scalable apps
6. Automated Testing JD: Jest/Karma/WDIO/Cucumber
7. Code Quality & Tooling JD: SonarQube/ESLint/CI
8. Scalable Front-End & Performance JD: scalable apps
9. Build Tooling & Bundlers Plus: Webpack/PNPM
10. Design Patterns & Architecture Misc
11. Web Fundamentals & Security Misc
12. Domain: Subscription / Payment Flows JD: product domain

1 · JavaScript Fundamentals

Reference vs Value

Primitives (string, number, boolean, null, undefined, symbol, bigint) are copied **by value**. **Objects/arrays/functions** are copied **by reference** — the variable holds a pointer to the object.

```
let a = 5; let b = a; b = 10; // a still 5 - b got a copy
const x = {v:1}; const y = x; y.v = 2; // x.v now 2 - same object, two refs
```

Equality follows the same rule: primitives compare by value (`5===5`), objects by identity (`{ }==={ }` is `false`).

React tie-in: React uses `Object.is` (reference check) on `setState`. Mutating an object keeps the same reference → React sees "no change" → no re-render. You must create a **new reference** (`{...obj}` , `[...arr]`). This is why you never mutate state.

Immutability

Not "a variable that can't be edited" (that's `const`). Immutability = **don't mutate existing data, create new copies with changes**.

```
setTodos([...todos, newTodo]);           // not todos.push()
setUser({...user, name: 'new'});         // not user.name = 'new'
setState(p => ({...p, f: {...p.f, s:1}})); // nested
```

"Where did you use immutability?" → State updates. I never mutate; I spread into a new reference. It's technically required because React compares references with `Object.is` — mutation keeps the same ref so no re-render fires. Same in Redux reducers, which also enables time-travel (each state is a distinct snapshot).

The Event Loop

```
console.log('1');
setTimeout(() => console.log('2'), 0);
Promise.resolve().then(() => console.log('3'));
console.log('4');
// → 1 4 3 2
```

Sync code runs first (1, 4). Then the **entire microtask queue drains** (promises → 3) before the next **macrotask** (setTimeout → 2). Microtasks (Promise.then, queueMicrotask) always beat macrotasks (setTimeout, events).

One-liner: `setTimeout(0)` doesn't mean "now" — it queues a new macrotask, and the whole microtask queue always drains before the next macrotask runs, so a promise callback always wins.

Closures & the var/let loop trap

```
for (var i=0; i<3; i++) setTimeout(() => console.log(i)); // 3 3 3
for (let i=0; i<3; i++) setTimeout(() => console.log(i)); // 0 1 2
```

`var` is function-scoped → one shared `i`; all callbacks read it after the loop ends (3). `let` is block-scoped and **re-binds per iteration** → each callback captures its own `i`. Closures capture the *variable*, not the value.

this binding & call/apply/bind

- `fn.call(ctx, a, b)` — invokes now, args listed individually
- `fn.apply(ctx, [a, b])` — invokes now, args as an **array** (A = Array)
- `fn.bind(ctx, a)` — does **not** invoke; returns a new bound function for later

Arrow functions have no own `this` — they inherit it from the enclosing scope (lexical `this`). Regular functions get `this` from how they're called.

Big O (spotting $O(n^2)$)

The everyday front-end fix: a `.find` / `.includes` inside a loop is secretly $O(n^2)$. Convert the array to a **Set/Map** for $O(1)$ lookups → $O(n)$.

```
// O(n^2)
items.filter(i => selectedIds.includes(i.id));
// O(n)
const sel = new Set(selectedIds);
items.filter(i => sel.has(i.id));
```

Ranking best→worst: $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2)$. Nested loop over same data = smell.

Map / Set / WeakMap / WeakSet

Type	Holds	Iterable / size	Refs	Use
Map	key→value (any key)	yes	strong	dict, cache, dedupe by key
Set	unique values	yes	strong	dedupe, fast membership
WeakMap	key→value (obj keys)	no	weak	per-object data, leak-free cache
WeakSet	unique objects	no	weak	tag/mark objects

Map keys are unique & compared by identity (two look-alike `{}` are different keys). `Map.has()` is $O(1)$ via hashing. Weak variants hold keys **weakly** → GC can reclaim them when nothing else references the key → prevents memory leaks; only take objects, aren't iterable.

Array.isArray

`Array.isArray(x)` — because `typeof [] === "object"` (useless) and `instanceof Array` breaks across iframe realms. `isArray` is realm-safe.

Debounce & Throttle

Debounce: wait for silence — fires once after calls stop (search input). **Throttle**: steady rate — fires at most once per interval during a burst (scroll, live feed).

```
function debounce(fn, delay) {  
  let timerId; // persists via closure  
  return function(...args) {  
    clearTimeout(timerId); // RESET on each call  
    timerId = setTimeout(() => fn.apply(this, args), delay);  
  };  
}
```

React gotcha: create the debounced fn **once** with `useMemo(() => debounce(fn, 300), [])` or `useRef`. Defining it in the render body makes a new closure (new `timerId`) every render, so it never debounces.

2 · TypeScript

type vs interface

- **interface** — extendable object shapes + **declaration merging** (declare twice → merges; used to augment 3rd-party/global types).
- **type** — the full type-algebra: **unions**, intersections, tuples, mapped & conditional types, primitive aliases.

Unique to type: **unions** (interfaces can't). Unique to interface: **merging**. Default: interface for object/prop shapes, type for unions & utilities.

Generics

```
function fetchJson<T>(url: string): Promise<T> {  
  return fetch(url).then(r => r.json());  
}  
  
const sub = await fetchJson<Subscription>('/api/sub'); // typed as Subscription
```

any vs unknown vs <T>: **any** = autocomplete but no safety (and it spreads). **unknown** = safe but unusable until narrowed. **<T>** = both — typed precisely per caller, reusable.

Type assertions (as)

x as string overrides the compiler — no conversion, no check. It moves risk from compile-time to runtime. Avoid on untrusted data.

Discriminated Unions (key for async/payment state)

```
type State =  
  | { status: 'loading' }  
  | { status: 'success'; data: Subscription }  
  | { status: 'error'; error: string };  
  
switch (state.status) {  
  case 'success': return state.data; // TS knows data exists here  
  case 'error':   return state.error; // TS knows error exists here  
}
```

The **discriminant** is a shared field with a unique **literal** value per branch (**status**). Checking it narrows the type. Keep the *whole union* in one state — impossible to have contradictory states (never loading+error at once). Accessing **state.data** in the loading branch is a compile error.

Senior note: keep the union in a single **useState<State>**, don't split into 3 booleans (**isLoading** / **isError** / **data**) — that reintroduces impossible states the union was preventing.

Runtime validation (Zod)

```
const Schema = z.object({ id: z.number(), plan: z.enum(['free', 'pro']) });  
type Sub = z.infer<typeof Schema>; // type FROM schema  
const user = Schema.parse(await res.json()); // validates at runtime, throws if wrong
```

Types are compile-time promises, not runtime guarantees. `fetchJson<Sub>` still *trusts* the server. Zod validates at the boundary and infers the type — the type is **earned**, not asserted. Critical for payment/subscription data you can't trust.

Useful utility types

`Partial<T>`, `Pick<T,K>`, `Omit<T,K>`, `Record<K,V>`, `Required<T>`, `ReturnType<F>`. Naming a few signals fluency.

3 · React

Why components re-render (3 causes)

1. Own **state** changes (`useState`).
2. **Context** it consumes changes.
3. **Parent** re-renders (child re-renders too — no comparison, unless wrapped in `React.memo`).

Diagnose with React DevTools **Profiler** (+ "record why each component rendered"), the **Highlight updates** toggle, and `why-did-you-render` for prop-identity churn.

Fix churn: `React.memo` the child + stabilize non-primitive props with `useMemo` / `useCallback` . Push state down / lift only as needed. For Context perf, move to a store with selectors (Zustand).

useEffect & dependency array

```
useEffect(() => {...}); // every render
useEffect(() => {...}, []); // once on mount
useEffect(() => {...}, [x]); // when x changes
// cleanup runs on unmount AND before each re-run:
useEffect(() => { const s = sub(); return () => s.unsubscribe(); }, []);
```

Empty `[]` = mount only. Omitted = every render. The returned cleanup = `componentWillUnmount` equivalent, and also runs before each re-run when a dep changes. Effects are for **synchronizing with external systems**, not "run once" logic — for a snapshot use `useState` / `useRef` , not an effect.

StrictMode: effects run twice on mount in dev to surface missing cleanup. Guard "fire once" with a ref, or move to a router event.

Lifecycle (class → hooks)

Class	Hook equivalent
<code>componentDidMount</code>	<code>useEffect(fn, [])</code>
<code>componentDidUpdate</code>	<code>useEffect(fn, [dep])</code>
<code>componentWillUnmount</code>	cleanup <code>return</code> from <code>useEffect</code>
<code>shouldComponentUpdate</code>	<code>React.memo</code>

Mental shift: hooks aren't about lifecycle *timing*, they're about **synchronizing** an effect with its dependencies.

Reconciliation & keys

React diffs lists by `key` to give elements **stable identity** across renders. Same key → reuse instance. **Index keys are dangerous:** on reorder/insert, React matches by position, so state/DOM (input text, focus, checkboxes) sticks to the wrong item. Index keys are only safe for static, append-only, *stateless* lists.

Built-in hooks (know these cold)

Core: `useState` , `useEffect` , `useContext` , `useRef` , `useMemo` , `useCallback` , `useReducer` .

Know of: `useLayoutEffect` (sync, before paint — DOM measure), `useTransition` / `useDeferredValue`

(responsiveness under heavy renders), `useSyncExternalStore` (safe external store subscribe), `useId` (SSR-safe ids).

Rules of Hooks: only call at top level (no conditions/loops) and only from React functions — React tracks hooks by call order, which must be identical every render.

Custom hooks & composition

Extract **stateful logic** (uses hooks/state) into a custom hook. **Stateless logic** → plain helper function, not a hook.

Shared **UI** → wrapper component via `children`.

Pitfall: each component calling a hook gets its *own* state — N components = N fetches. To share, lift/Context/React Query (dedupes by key).

Fragments

`<>...</>` groups children without adding a DOM node. Use `<React.Fragment key={...}>` when you need a key in a list.

Memory leaks (distinct types)

- **Uncleaned effect resources** — listeners/timers/subscriptions/in-flight fetches. Fix: cleanup return (`removeEventListener` , `clearInterval` , `unsubscribe` , `AbortController.abort()`).
- **Detached DOM nodes** held in refs after removal. Fix: null the ref; WeakMap for node-keyed data.
- **Unbounded growth** — module-level cache/array that only grows. Fix: eviction (TTL/LRU), size caps.
- **Closure over-capture** — long-lived callback pins a big object.
- **Library instances** — chart/map/editor needing their own `.destroy()` .
- **Strong-ref caches** keyed by objects → use WeakMap.

Find them: DevTools Memory tab, heap snapshot diff across mount/unmount.

Function vs Class components (modern React)

Default to **function components**. Classes are legacy — all new APIs are function-only. The **only** remaining reason for a class is an **error boundary** (`static getDerivedStateFromError` + `componentDidCatch` — no hook equivalent yet; usually one boundary or the `react-error-boundary` lib).

API	What it is
Suspense	show a fallback while children (code or data) aren't ready
<code>use</code> hook	read a Promise or Context during render; can be called conditionally
Actions	async form/mutation handling with built-in pending state (<code>useActionState</code> , <code>useFormStatus</code> , <code>useOptimistic</code>)
Server Components	render on server, ship no JS to browser; direct backend access
React Compiler	build-time auto-memoization (no manual <code>useMemo</code> / <code>useCallback</code>)

React Fiber

React's **reconciliation engine** (rewritten in React 16). It reimplemented rendering as **incremental and interruptible** — work is split into units ("fibers", one per element) that React can pause, prioritize, resume, or discard, instead of rendering the whole tree in one blocking pass.

Problem it solved: old reconciliation was synchronous/recursive — a big update blocked the main thread until done (janky UI). Fiber makes rendering yieldable.

One-liner: Fiber is React's reconciliation engine (React 16) that made rendering incremental and interruptible — it splits work into units it can pause, prioritize, and resume, powering concurrent rendering, Suspense, and transitions. It's an internal rewrite; component code didn't change. The Virtual DOM is *what* React diffs; Fiber is the *engine* that diffs it.

Main thread: what React renders vs what the browser renders

JS and browser painting share ONE thread — they can't run at the same time. While JS runs, the browser can't paint or handle input.

- **React "render"** = JavaScript work: run your components, diff the fiber tree, compute what DOM changes are needed. No pixels yet.
- **Browser "render/paint"** = turn the DOM into actual pixels (layout → paint → composite), plus handling input and animations.

"Yielding to the browser" = React pauses its JS work and hands the thread back so the browser can paint a frame and process input, then resumes. This is how time-slicing keeps the UI responsive.

Priority (lane model)

React assigns each update a priority ("lanes") and processes higher priority first. It **infers** priority from the source (you don't set it, except via transitions):

1. **Immediate/sync** — `flushSync`
2. **User-blocking** — input, click, hover (user waits)
3. **Normal** — data arrived, render it
4. **Transition/low** — marked with `startTransition`
5. **Idle** — analytics, prefetch

If an urgent update arrives while a low-priority render is in progress, Fiber **interrupts** it, handles the urgent one, then resumes (or discards) the deferred work.

Transitions (useTransition / startTransition)

Mark an update as **non-urgent and interruptible** so urgent updates (typing) aren't blocked by heavy ones (filtering a huge list).

```
function onChange(e) {
  setQuery(e.target.value); // URGENT – input updates instantly
  startTransition(() => {
    setResults(filterHugeList(e.target.value)); // NON-URGENT – interruptible
  });
}
// isPending flag shows the transition is in progress
```

It doesn't make the render faster — it makes it **low-priority and interruptible**. If the user types again mid-render, React discards the unfinished list render and starts over. `useDeferredValue` = same idea for a *value* you receive rather than a setter you own.

Time-slicing

The mechanism Fiber uses to **split rendering into small chunks and spread them across frames**, yielding to the browser between chunks so the main thread isn't blocked longer than a frame (~16ms for 60fps).

```
Without: [===== 100ms render =====] browser frozen the whole time
With:    [8ms][→browser][8ms][→browser][8ms]... browser paints & handles input between
```

How the pieces relate: **time-slicing** = the mechanism (chunk + yield); **lanes** = the policy (what to do first); **transitions** = your API to mark work low-priority. Fiber laid this in React 16; **concurrent rendering actually turned on in React 18** (`createRoot`).

flushSync

`flushSync` (from `react-dom`) forces React to apply an update **synchronously and immediately**, repainting the DOM before the next line — it disables batching. Highest (sync) priority; the *opposite* of transitions.

```
flushSync(() => { setItems([...items, newItem]); }); // DOM updated NOW
listRef.current.lastChild.scrollIntoView();           // safe: row exists in DOM
```

Use rarely — only when you must read the updated DOM or control focus/scroll right after a state change. It hurts performance (forces sync render, disables batching/concurrency), so it's an escape hatch.

Suspense data fetching & the use hook

Trigger: a component "suspends" by **throwing a Promise** when data isn't ready. React catches it, shows the nearest `<Suspense>` fallback, subscribes to the Promise, and re-renders when it resolves. (Rejections → Error Boundary.)

```
// use() reads a Promise (or Context)
function Profile({ userPromise }) {
  const user = use(userPromise); // pending → throws (suspends); resolved → returns value
  return <p>{user.name}</p>;
}
```

Stable promise required: `use` identifies a Promise by **reference**. Creating it in the component body (`const p = fetch()`) makes a NEW promise every render → React never sees it resolve → infinite suspend + request flood.

The promise must come from a stable source: **a Server Component**, a `useRef`, or **React Query's cache** (`useSuspenseQuery`).

use is not the only way to suspend — any Promise-throw triggers Suspense. In practice you rarely pass a raw promise: use `useSuspenseQuery` (client) or an async Server Component. `use` also reads **Context** and — unlike other hooks — can be called **conditionally** (it holds no per-slot state, so it isn't bound to call order). `use(Context)` is niche: only useful to read context after an early return where the branch condition is computed inside the component; otherwise plain `useContext` at the top is simpler.

Server Components: async data fetching

Server Components can be **async functions** and `await` data *in the body* — the fetch runs on the server before render, the client gets ready HTML (no loading state), and the fetch code never ships to the browser.

```
async function DashboardPage() { // Server Component (default in App Router)
  const user = await fetchUser(1); // await directly — runs on server
  return <h1>Welcome {user.name}</h1>;
}
```

Wrap in `<Suspense>` for **streaming**: the fast part of the page shows immediately with a fallback, and the slow part streams in when its `await` resolves. Data-fetching options, modern → old: **RSC async/await** (App Router default) → **use(promise)** (pass a server promise to a client component) → **React Query / useEffect+fetch** (client-side, realtime or interaction-driven).

Code splitting & React.lazy

`React.lazy` is Suspense for **component code** (not data) — the component loads as a separate chunk on demand; Suspense shows a fallback while the JS downloads.

```
const HeavyChart = lazy(() => import('./HeavyChart'));  
<Suspense fallback={<Spinner/>}><HeavyChart/></Suspense>
```

Next.js: route-level splitting is **automatic** — each **page.tsx** is its own chunk, loaded on navigation (with prefetch of visible links). You do NOT wrap whole routes in lazy. Use **next/dynamic** only for heavy components *inside* a route (charts, editors, modals). Manual **React.lazy** for routes is for plain React (Vite/CRA) without Next.

4 · RxJS

Observable vs Promise

	Promise	Observable
Values	one	0 to many (stream)
Starts	on creation (eager)	on subscribe (lazy)
Per consumer	once, shared	re-runs per subscriber
Cancellable	no	yes (unsubscribe)
Operators	.then	full pipeline
Terminal	resolve/reject	complete/error

One-liner: A Promise is a single eager future value; an Observable is a lazy, cancellable stream of many values that starts on subscribe, re-runs per subscriber, and composes with operators.

Unsubscribing (avoid leaks)

Angular: **takeUntil(destroy\$)** for ongoing, **take(1)** for one-shot, async pipe. **React (this role):** subscribe in **useEffect**, unsubscribe in cleanup.

```
useEffect(() => {
  const sub = source$.subscribe(setState);
  return () => sub.unsubscribe(); // React equiv of takeUntil / ngOnDestroy
}, []);
```

Flattening operators (know the contrast — payment domain!)

Operator	On new value while busy	Use case
switchMap	cancel old, start new	typeahead / "latest wins"
mergeMap	run all in parallel	independent concurrent ops
concatMap	queue , in order	ordered sequential writes
exhaustMap	ignore new until done	payment submit (no double-fire)

The trap: Search → **switchMap** (cancel stale, avoid out-of-order overwrite). Payment submit → **exhaustMap** (ignore duplicate clicks). Using **switchMap** for payment is dangerous — a 2nd click cancels the 1st charge mid-flight; **mergeMap** → double charge. Rule: cancel is fine for reads, dangerous for writes.

Typeahead search — full pipe

```
keyup$.pipe(
  map(e => e.target.value),
  filter(q => q.length >= 2),           // skip too-short
  debounceTime(300),                   // wait for pause
  distinctUntilChanged(),               // skip identical consecutive
  switchMap(q => ajax.getJSON(`/search?q=${q}`).pipe(
    catchError(() => of([]))             // INNER catch – outer stream survives
  ))
).subscribe(render);
```

Subjects

A **Subject** is both Observable and Observer (you can `.next()` into it) and is **multicast** (one execution shared by all subscribers), vs a plain Observable which is unicast.

Type	Emits to new subscriber
Subject	nothing (only future emissions)
BehaviorSubject	current/latest value (needs initial) — state
ReplaySubject	last N values (history)
AsyncSubject	only final value, on complete

Hot vs Cold

Cold = unicast, producer created per subscribe (HTTP Observable → 2 subscribers = 2 requests). **Hot** = multicast, shared. Convert cold→hot with `share()` or `shareReplay(1)` (multicast + cache last value for late subscribers) — the standard fix to dedupe an HTTP call. Backed by a Subject internally.

Combination operators

Operator	Emits	Use
combineLatest	latest of each, on any emission	reactive UI from multiple streams
merge	each value as it arrives	combine independent event sources
forkJoin	each source's last value, when all complete	parallel API calls, render when all done (RxJS Promise.all)
zip	values paired by index	strict positional pairing (rare)

Error handling & retry

`catchError` returns a **replacement Observable** (not a throw). **Placement matters**: an error kills the stream it reaches — catch *inside* `switchMap` 's inner Observable so a failed request returns `of([])` and the outer search stream survives; catching outside kills the whole input. `retry` re-subscribes to the source; place it **before** `catchError`.

```
http.get(url).pipe(
  retry({ count: 3, delay: (e,n) => timer(n*1000) }), // backoff
  catchError(() => of(fallback))
)
```

When RxJS in React?

Answer: React Query for server fetching, `useState` / Zustand for client state. RxJS when the hard problem is **coordinating streams of events over time** — debounce + cancel + merge + throttle across multiple async sources, or real-time push (WebSocket). Not for a single fetch.

5 · State Management & Data Fetching

Decision tree

1. **Server data?** → React Query / SWR (cache, refetch, dedupe — don't hand-roll in Redux).
2. **Local UI state?** → `useState`, lifted as needed.
3. **Cross-tree, low-frequency?** → Context (theme, auth, locale).
4. **High-frequency or complex client state?** → Zustand / Redux (selector subscriptions).

Context is distribution, not management — no selectors, every consumer re-renders on any change. Best for low-frequency values. High-frequency state (keystrokes) in Context = perf problem → use a store with selectors.

React Query (TanStack Query)

Server-state manager: treats server data as a **cache with lifecycle** (stale, refetch, invalidate), not `useState`. Gives caching by `queryKey`, request dedup, background refetch (on focus/reconnect/interval), `isFetching` vs `isLoading`, mutations + invalidation, optimistic updates, and out-of-order race handling — all the boilerplate you'd otherwise hand-roll around `useEffect` + `fetch`.

Zustand

```
const useStore = create<S>((set) => ({
  count: 0,
  increment: () => set((s) => ({ count: s.count + 1 })),
}));
// component — SELECT only what you need (perf):
const count = useStore((s) => s.count);
```

External store, no provider, selectors mean components re-render only on the slice they use (vs Context). `set` **shallow-merges** top-level. Readable outside React via `getState()` — good for bridging an RxJS stream into the store.

persist middleware

```
create(persist(
  (set) => ({ ... }),
  { name: 'checkout', storage: createJSONStorage(() => sessionStorage),
    partialize: (s) => ({ plan: s.plan }) } // only persist chosen fields
));
```

Auto-saves to local/sessionStorage under `name`, rehydrates on load. `partialize` controls what's saved (never persist card data). `devtools` middleware = Redux DevTools + time-travel.

Redux Toolkit

Official modern Redux — `createSlice` colocates state+reducers+actions (auto-generates action creators), Immer lets you write "mutating" syntax safely, `configureStore` gives DevTools + middleware. Same niche as Zustand (client state) but more structured (slices, dispatch, provider) — favored by large teams. Includes RTK Query for server state.

Client state: RTK / Zustand. Server state: React Query / RTK Query.

6 · Automated Testing

Strategy — the testing trophy

React weights the **most** on integration tests (best confidence per test), thin unit + E2E layers.

- **Unit** (Jest/Vitest) — pure functions, hooks, reducers, utils.
- **Integration** (Jest + RTL) — component + children + hooks together. The bulk.
- **E2E** (WebdriverIO/Playwright) — full flow in a real browser. Fewest, for money paths.

React Testing Library philosophy

Principle: "The more your tests resemble how the software is used, the more confidence they give." Query by role/label/text, assert what the **user sees** — never internal state or implementation. Rule: refactor internals without changing behavior → test still passes.

What RTL exposes

- **render** — mount into jsdom. **screen** — query the DOM.
- **Query variants:** **getBy** (throws if missing), **queryBy** (returns null — for absence), **findBy** (async, waits). Plus **getAllBy** etc.
- **Selectors (priority):** **ByRole** > **ByLabelText** > **ByText** > **TestId** (escape hatch).
- **userEvent** (realistic interactions, preferred) vs **fireEvent** (single raw event).
- **waitFor** (async assertion), **renderHook** (custom hooks), **within** (scoped queries).

Matchers (**toBeInTheDocument**) come from **jest-dom**; **expect** / **toBe** from Jest — not RTL. RTL finds & interacts; Jest asserts.

Implicit roles & accessible name

getByRole('link', { name: 'X' }) works because `<a href>` has the implicit role **link** and its text is the accessible name. Querying this way = testing the UI exactly as assistive tech sees it. If **getByRole** can't find it, that's a signal the element is inaccessible.

Test the network boundary

```
// mock a module (e.g. Next router)
jest.mock('next/navigation', () => ({ usePathname: jest.fn() }));
(usePathname as jest.Mock).mockReturnValue('/tournaments');

// async fetch component — findBy waits
expect(await screen.findByText('Pro Plan')).toBeInTheDocument();
```

Mock the **network** (MSW or mocked fetch), not internals. Test success + error + loading. Arrange (mock, render) → Act (interact) → Assert (**expect**).

jsdom

A JS implementation of the DOM that runs in Node — gives tests **document** / **window** / elements without a real browser. Fast, headless, but simulates DOM *structure* without real rendering/layout — which is why E2E still needs a real browser.

The stack (this JD)

- **Jest** — unit + integration runner (+ RTL for components). Next.js: use **next/jest** config helper.

- **Karma** — older runner in real browsers (Angular-heritage signal).
- **WebdriverIO + Cucumber** — E2E with **BDD/Gherkin**: business-readable Given/When/Then scenarios, backed by step-definition functions driving the browser. Used for critical flows (checkout/payment).

BDD = tests as plain-language behavior specs, doubling as documentation. Gherkin = the Given/When/Then syntax. Cucumber = runs Gherkin by mapping each step to code.

7 · Code Quality & Tooling

ESLint & SonarQube

- **ESLint** — static analysis for JS/TS: catches bugs, enforces style/rules, plugin ecosystem (react-hooks rules enforce Rules of Hooks / exhaustive-deps). Runs in editor + CI + pre-commit.
- **SonarQube** — deeper static-analysis platform: code smells, bugs, security vulnerabilities, duplication, coverage gates, tech-debt tracking across the whole codebase. Often a CI quality gate that fails the build.
- **Prettier** — formatting (separate from linting). Push formatting into tooling so it's not a review topic.

Code review (senior approach)

Priority order: **correctness** → **design/maintainability** → **tests** → **style (automate)**. Comment on logic bugs, wrong abstractions, consistency, test coverage. Let personal-preference style slide. Separate **blocking** from **nit:** (non-blocking). Frame as questions, praise the good, take design arguments to a call.

CI/CD & Git tooling Plus

- **CI/CD** — pipeline runs lint, type-check, tests, build on every push; fails fast; deploys on merge. Add a **bundle-size budget** that breaks CI.
- **GitLab CI** — `.gitlab-ci.yml` defines stages (build/test/deploy) as jobs. Same concept as GitHub Actions.
- **Lighthouse CI** — fails build if perf/Core Web Vitals regress.

SOLID (quality principles)

Principle	Meaning
S — Single Responsibility	one reason to change (split data logic from render)
O — Open/Closed	extend without modifying (new payment strategy = new class)
L — Liskov Substitution	any implementation substitutable for its interface
I — Interface Segregation	many narrow interfaces > one fat one
D — Dependency Inversion	depend on abstractions, not concretions

In React, applied in spirit: separate responsibilities (hooks), depend on abstractions (Strategy for payment methods → OCP + DIP).

8 · Scalable Front-End & Performance

Rendering strategies (2 axes: static ↔ dynamic, shared ↔ per-user)

Strategy	HTML generated	Use for
CSR	in browser at runtime	private interactive apps (dashboard, inbox)
SSR	per request on server	fresh/personalized + SEO (product page, feed)
SSG	at build time	same for everyone, rarely changes (landing, docs, blog)
ISR	build + periodic revalidate	mostly static, needs periodic freshness (news, listings)

Rule: generate as early as you can afford — build time if content allows, request time only if it must be fresh, client-side only if private/interactive. Both SSR and SSG hydrate; the difference is *when* HTML is made.

Bundle size / performance levers

Diagnose first: bundle analyzer ([@next/bundle-analyzer](#)), Lighthouse, Coverage tab.

1. **Code splitting** (biggest lever) — route-level auto in Next; manual with `next/dynamic` / `React.lazy` + `Suspense` for heavy components (charts, editors, modals). Best: load on interaction.
2. **Dependency audit** — replace heavy libs (moment→date-fns), import named exports for tree-shaking, kill duplicates.
3. **RSC boundary** — keep `'use client'` low; Server Components never ship JS to the browser.
4. **Assets** — Brotli/gzip, image optimization, font subsetting.

Set up guardrails day one: bundle-size budget that breaks CI, Lighthouse CI, analyzer wired into the build. The amateur optimizes when slow; the senior prevents silent regressions.

Code splitting example

```
const Chart = dynamic(() => import('./Chart'), { ssr: false });
// or plain React:
const Chart = lazy(() => import('./Chart'));
<Suspense fallback={<Spinner/>}><Chart/></Suspense>
```

Core Web Vitals (quick)

LCP (loading — largest element paint; fix: optimize images/fonts, SSR, preload). **INP/FID** (interactivity — input responsiveness; fix: reduce/split JS, defer). **CLS** (visual stability — layout shift; fix: set dimensions on images/embeds, reserve space).

9 · Build Tooling & Bundlers Plus

The build pipeline

You write modern TS/JSX → **transpile** (SWC/Babel) to browser-compatible syntax + **polyfill** missing APIs (core-js) per browserslist → **tree-shake** dead code → **minify + mangle** (Terser) → **source maps** for debugging → CDN **Brotli/gzip** compresses for transfer.

- **Polyfill** — adds missing *features/APIs* (Promise, fetch). **Transpile** — converts newer *syntax* to older. Both driven by browserslist.
- **Minify** — shrink source (whitespace). **Compress (gzip/Brotli)** — shrink bytes on the wire. They stack.
- **Tree-shaking** — drop unused exports; needs ES modules.
- **Source maps** — map minified code back to source for debugging; don't expose publicly.

In modern Next.js all of this is automatic (SWC). The only lever you'd tune is **browserslist** — over-broad targets bloat polyfills. Knowledge is for debugging/tuning, not manual setup.

Webpack / Vite / SWC / PNPM

- **Webpack** — mature, configurable bundler (module graph, loaders, plugins, code splitting). Powers Next under the hood.
- **Vite** — modern, fast (esbuild dev, Rollup build); default for new non-Next apps.
- **SWC** — Speedy Web Compiler (Rust), replaced Babel in Next; ~20–70× faster transpile/minify.
- **PNPM** — fast, disk-efficient package manager: content-addressable store + hard links (one copy of a version on disk), strict non-flat `node_modules` that prevents phantom deps. Great for monorepos.

10 · Design Patterns & Architecture Misc

Composition over inheritance

Components aren't classes → no inheritance. Compose instead: shared **UI** via `children` /slots/wrappers; shared **logic** via custom hooks (stateful) or helper functions (stateless).

Patterns with React use cases

Pattern	Use in React
Compound components	flexible APIs (Tabs/Accordion) sharing state via Context
Provider pattern	Context provider for cross-cutting state (theme, auth)
Custom hooks	share stateful logic (modern replacement for HOC/render props)
HOC	wrap component to inject behavior (<code>withAuth</code>) — older
Render props	pass a function as children — older
Container/Presentational	separate data logic from render (now via hooks)
Observer	RxJS <i>is</i> this; Zustand/TanStack selectors
Factory	create different component by type (form-field builder)
Strategy	swap behavior by type (payment methods behind one interface)
Reducer	complex state via actions (useReducer / Redux) — checkout flow

Singleton ≠ Context. Context = Provider pattern (allows multiple providers). Singleton = one instance (a module-level store/client). Both Context & Zustand can be instantiated multiple times → neither is inherently a Singleton.

Strategy example (payment)

```
interface PaymentStrategy { pay(amount: number): Promise<Result>; }
const strategies = {
  card: (a) => stripe.charge(a),
  paypal: (a) => paypal.createOrder(a),
};
function checkout(method, amount) { return strategies[method](amount); }
// new method = new entry, checkout() unchanged (OCP + DIP via the interface abstraction)
```

Factory decides *what to create*; Strategy decides *what behavior to run*. Often used together.

Container / Presentational example

```
// presentational – pure, props only
function UserList({ users, isLoading }) { return isLoading ? <Spinner/> : <ul>...</ul>; }
// container role now lives in a hook:
function Users() { const { data, isLoading } = useUsers(); return <UserList .../>; }
```

11 · Web Fundamentals & Security Misc

HTTP request vs response

Same structure (start line, headers, optional body), opposite direction. **Request:** method + path (+ body for POST/PUT). **Response:** status code + data.

Status classes: 2xx success (200, 201, 204) · 3xx redirect (301, 304) · 4xx **client** error (400, 401 not authenticated, 403 forbidden, 404, 429) · 5xx **server** error (500, 502, 503). 401 = "who are you"; 403 = "you can't do this".

Storage: localStorage / sessionStorage / Cookies

	localStorage	sessionStorage	Cookies
Lifespan	until cleared	until tab closes	configurable
Across tabs	yes	no (per-tab)	yes
Size	~5–10MB	~5MB	~4KB
Sent to server	no	no	every request

Auth tokens: NOT in localStorage (any XSS can read it). Use **HttpOnly, Secure, SameSite cookies** — HttpOnly means JS can't read it (blocks XSS token theft), SameSite blocks CSRF. Cookies auto-send on every request, so keep them small; scope with **Path**; serve static assets cookieless.

XSS vs CSRF (opposite attacks, opposite defenses)

- **XSS** — attacker runs *their* JS in the victim's browser in your origin (stored comment, reflected URL, unsafe DOM write). Can read data/cookies. **Fix:** escape output (React auto-escapes JSX; risk is `dangerouslySetInnerHTML` → sanitize with DOMPurify), CSP header, HttpOnly cookies.
- **CSRF** — attacker makes the victim's browser send an authenticated request using auto-attached cookies (hidden form/image). Can't read anything, only forge actions. **Fix:** **SameSite** cookies + CSRF tokens.

Why HttpOnly + SameSite cookies are the auth default: HttpOnly stops XSS reading the token, SameSite stops CSRF sending it cross-site — both defended in one mechanism.

12 · Domain: Subscription / Payment Flows

Multi-step checkout architecture

1. **State:** multi-step wizard, central **Zustand** store (high-frequency, multi-field → selectors beat Context), persisted to **sessionStorage** via **persist**. *Never persist card data (PCI).*
2. **Navigation:** step in URL (path or query — query fine if you fire **page_view** per step for analytics) for back/forward + refresh. **Route guards** redirect to step 1 if prerequisite state is missing (shared/bookmarked link).
3. **Validation:** per-step gates progression (Zod + React Hook Form), final validation before submit, and **server re-validates** (client validation is UX, not security).
4. **Payment: idempotency key** (client-generated UUID; server dedupes duplicate submits — safe to retry on timeout).
Front-end guard: **exhaustMap** / disabled button + **isSubmitting**. Card data goes straight to Stripe, never your store.

Step with Zod + React Hook Form

```
const { register, handleSubmit, formState:{errors} } = useForm({
  resolver: zodResolver(accountSchema), // Zod validates
  defaultValues: account,              // restore on back-nav/refresh
});
// handleSubmit won't call onSubmit if invalid → per-step gate, automatic
```

Idempotent submit

```
const idempotencyKey = useRef(crypto.randomUUID()); // stable across retries
const submit = async (data) => {
  if (isSubmitting) return; // front guard
  const res = await fetch('/api/checkout', {
    method: 'POST',
    headers: { 'Idempotency-Key': idempotencyKey.current }, // server dedupes
    body: JSON.stringify(data),
  });
};
```

Feature folder

```
features/checkout/
  components/ (CheckoutWizard, steps/, StepIndicator)
  store/      (checkoutStore.ts – Zustand + persist)
  schemas/    (Zod schemas per step)
  hooks/      (useCheckoutSubmit.ts)
```

Interview meta-advice (from feedback): the gap was *precision*, not knowledge. Three fixes: (1) **Name things** — "composition over inheritance", "discriminated union", "copied by reference", "idempotency key". (2) **Pair concept + use case** — never just define; always "and I'd use this for X". (3) **Lead with the crisp one-liner**, then elaborate if probed. Don't ramble toward the answer.